

Addis Ababa university **AAiT** campus programing (python) final exam with perfect answer

Part one

Error finding (10 pts.)

1. `number = input("please enter any number: ")` # the number can not be interpreted as an integer.
2. `For i in range(1,number):`
3. `if (number % i ==0):`
4. `sum += 1` # unsupported operand types(s)
5. `if (sum == number):`
6. `print(number , "is a perfect number")`
7. `else:`
8. `print(number , "is not a perfect number")`

Answer

Line number	Error type	solution
1	Type error	Number = int(input("Enter number:n"))
4	Type error	Sum =number +1

Question 2

```
for (i in range(11.5)):  
    print(i, end=' ')
```



Line error	Error type	solution
1	Syntax error	for i in range float(11.5)

Question 3

```
i +=2
```

```
while(i <== 9): # assignment operator syntax error
```

```
    print('the value of i is:')
```

```
Print(i+=1)
```

Answer

Line number	Error type	solution
2	Syntax error	while(i<= 9):
1	Name error	i is not defined

Error type

Description

Index Error

When the wrong index of a list is retrieved.

AssertionError

It occurs when the assert statement fails

AttributeError

It occurs when an attribute assignment is failed.

ImportError

It occurs when an imported module is not found.

Key Error

It occurs when the key of the dictionary is not found.

NameError

It occurs when the variable is not defined.

MemoryError

It occurs when a program runs out of memory.

Type Error

It occurs when a function and operation are applied in an incorrect type.

Common error finding method in python

1) Forgetting to put a :

- at the end of an if, elif, else, for, while, class, or def statement.
- (Causes “Syntax Error: **invalid syntax**”)

Example:-

```
For i in Range(10):  
    if(i==5)  
        break:  
    else: print(i)  
        continue
```

2) Using = instead of ==.

➤ (Causes “Syntax Error: invalid syntax”)

```
a, b = 0  
if (a = b)  
    a + b = c  
    print( z)
```

3) Using the wrong amount of indentation.

- ✓ (Causes “Indentation Error: unexpected indent” and “Indentation Error: unindent does not match any outer indentation level” and “Indentation Error: expected an indented block”)
- ❖ the indentation only increases after a statement ending with
 1. a : colon, and
 2. afterwards must return to the previous indentation.

- 
- 4) Forgetting the `len()` call in a for loop statement.
- (Causes “`TypeError: 'list' object cannot be interpreted as an integer`”)
 - Commonly you want to iterate over the indexes of items in a list or string, which requires calling the `range()` function.
 - Just remember to pass the return value of `Len(some List)`, instead of passing just some List.

5) Trying to modify a string value.

- (Causes “TypeError: 'str' object does not support item assignment”)
- Strings are an immutable data type.



6) Trying to concatenate a non-string value to a string value.

- (Causes “Type Error: Can't convert 'int' object to str implicitly”)

7) Forgetting a quote to begin or end a string value.

➤ (Causes “Syntax Error: EOL while scanning string literal”)

Example1:-

```
str = “Educational channel please subscribe  
for c in range[3,9]  
    Print (str(c))
```

Example2:

```
a = input("Enter any number")  
if a%2=0:  
    print("Even number")  
Else :  
    print("Odd number")
```



8) A typo for a variable or function name.

➤ (Causes “Name Error: name 'fooba' is not defined”)



9) A typo for a method name.

(Causes “Attribute Error: 'str' object has no attribute 'lowerr'”)



10) Going past the last index of a list.

➤ (Causes “Index Error: list index out of range”)



11) Using a non-existent dictionary key.

➤ (Causes “Key Error: 'spam'”)

- 
- 12)** Trying to use a Python keyword for a variable name.
- (Causes “Syntax Error: invalid syntax”)



13) Using an augmented assignment operator on a new variable.

- (Causes “Name Error: name 'foobar' is not defined”)
- Do not assume that variables start off with a value such as 0 or the blank string.
- A statement with an augmented operator like `spam += 1` is equivalent to `spam = spam + 1`.
- This means that there must be a value in `spam` to begin with.



14) Using a local variable (with the same name as a global variable) in a function before assigning the local variable.

- (Causes “UnboundLocal Error: local variable 'foobar' referenced before assignment”)
- * Using a local variable in a function that has the same name as a global variable is tricky. The rule is: if a variable in a function is ever assigned something, it is always a local variable when used inside that function. Otherwise, it is the global variable inside that function.
- * This means you cannot use it as a global variable in the function before assigning it.



15) Trying to use `range()` to create a list of integers.
(Causes “Type Error: 'range' object does not support item assignment”)

- Sometimes you want a list of integer values in order, so `range()` seems like a good way to generate this list.
- However, you must remember that `range()` returns a "range object", and not an actual list value.



16) There is no ++ increment or -- decrement operator.
(Causes “SyntaxError: invalid syntax”)

- If you come from a different programming language like C++, Java, or PHP, you may try to increment or decrement a variable with ++ or --.
- There are no such operators in Python.



Part two

Code output(10pt.)

```
4. row = 6
String = "python"
for i in range(row,0,-1):
    j = 0
    for k in string:
        if(j == i):
            break
        print(j,end=k)
        j += 1
    print(" ")
```

Question 5

```
def output(X(i)):  
    i += 1  
    print(i)  
    i = 19  
for i in range(10):  
    output(X(i))  
    i+=5  
output(X(i))
```

Part3

Completion(14pt.)

Fibonacci series

The first number start with 0 and the second number start with 1

```
First,second=0,1
```

```
n= t-in-----
```

```
Print("Fibonacci series are: ")
```

```
For i in--range(n):-----
```

```
    if i <= 1:
```

```
        result = i
```

```
    else:
```

```
        result =first + second-----
```

```
        first =--second-----
```

```
        second=-result-----
```

```
        print(result)
```

Fibonacci Series In Python

- Fibonacci series in python is a sequence of numbers in which the current term is the sum of the previous two terms.
- The first two terms of the Fibonacci sequence are 0 and 1.
So what is the logic behind this?
- Let's understand it in detail.

The logic behind Fibonacci sequence in python

- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597,... this is the Fibonacci sequence.
- As you can see, the first two terms of the sequence are 0 and 1.
- The third term is an addition to the first two terms and so on.
 - ✓ In general,
 - ✓ $F_n = F_{n-1} + F_{n-2}$
 - ✓ Where F_n is a current term.

Python Five method to Fibonacci series

Table of Contents

1. Fibonacci series in python using for loop
2. Fibonacci series in python using while loop
3. Fibonacci series using recursion in python
4. Printing Nth term of Fibonacci series using recursion
5. Printing Fibonacci series dynamically

1. Fibonacci series in python using for loop

- *Accept the number of terms from the user and pass that number in the loop and implement the Fibonacci number's logic in the loop.*

```
n = int(input("Enter number of terms: "))
n1, n2 = 0, 1
i = 0
if n <= 0:
    print("Please enter a positive integer")
elif n == 1:
    print("Fibonacci sequence up to",n,":")
    print(n1)
else:
    print("Fibonacci sequence:")
    for i in range(n):
        print(n1)
        sum = n1 + n2
        n1 = n2
        n2 = sum
        i += 1
```

Fibonacci series in python using while loop

- Take a number of terms of the Fibonacci series as input from the user and iterate while loop with the logic of the Fibonacci series.

```
n = int(input("Enter number of terms: "))
n1, n2 = 0, 1
i = 0
if n <= 0:
    print("Please enter a positive integer")
elif n == 1:
    print("Fibonacci sequence up to",n,":")
    print(n1)
else:
    print("Fibonacci sequence:")
    while (i < n):
        print(n1)
        sum = n1 + n2
        n1 = n2
        n2 = sum
        i += 1
```

3.Fibonacci series using recursion in python

- Same logic as above, accept a number from the user and pass it to the function.
- The function will return the value and outside we print it.

```
def Fibonacci(n):          #fibonacci function
    if n < 0:
        print("Incorrect input")
    elif n == 0:
        return 0
    elif n == 1 or n == 2:
        return 1
    else:
        return Fibonacci(n - 1) + Fibonacci(n - 2) #Logic for the fibonacci series
n = int(input("Enter number of terms: "))
i = 0
for i in range(n):
    print(Fibonacci(i))
```

4. Printing Nth term of Fibonacci series using recursion

- ❖ Accept a number of a term from the user and pass it to the function.
- ❖ It will return the exact term of the Fibonacci series.

```
def Fib(x): #fibonacci function
    if x < 0:
        print("Incorrect input")
    elif x == 0:
        return 0
    elif x == 1 or x == 2:
        return 1
    else:
        return Fib(x-1) + Fib(x-2) #Logic for the fibonacci series
x = int(input("Enter number of terms: "))
print(Fib(x))
```

5. Printing Fibonacci series dynamically

- Initialize an array with 0 and 1 and accept a number of terms from the user.
- The function will append the values to the array which will reduce memory wastage.

```
def fib(n):  
    a = [0,1]  
    if n == 1:  
        print('0')  
    elif n == 2:  
        print(a)  
    else:  
        while(len(a) < n):  
            a.append(0)  
            if(n == 0 or n == 1):  
                return 1  
            else:  
                a[0]=0  
                a[1]=1  
                for i in range(2, n):  
                    a[i]=a[i - 1]+a[i - 2]  
                print("First {} terms of fibonacci series:".format(n),a)  
                return a[n - 2]  
n = int(input("Enter number of term: "))  
fib(n)
```

Question7

check Armstrong number

```
digit = 0
result = 0
number = 370
number1 = number
temp = number
#digit counting code
while number != 0:
    number =--number//10-----
#the sum of each digit powered to the digit count
While-number1!=0-----:
    number=-----
    result=-----
    number1 = number1//10
If-----:
    print("number is Armstrong")
else:
    Print("number is not Armstrong")
```

Python Program to Check Armstrong Number

- * In this tutorial, we are going learn to check whether an n-digit integer is an Armstrong number or not.
- * To understand this concept, you should have the knowledge of the following [Python programming](#) topics:
 1. [Python if...else Statement](#)
 2. [Python while Loop](#)

Concept of Armstrong number

- * A positive integer is called an Armstrong number of order n if the sum of the cube of the each digit is equal to it.
- * For example if a number is abcd.... in order to become armstrong number it must satisfies the following condition

$$abcd... = a^n + b^n + c^n + d^n + \dots$$

- * In case of an Armstrong number of 3 digits, the sum of cubes of each digit is equal to the number itself. For example:
- * $153 = 1*1*1 + 5*5*5 + 3*3*3$ // 153 is an Armstrong number.

Python code to check whether a given number is Armstrong number or not

```
# take input from the user
num = int(input("Enter a number: "))

# initialize sum
sum = 0

# find the sum of the cube of each digit
temp = num
while temp > 0:
    digit = temp % 10
    sum += digit ** 3
    temp //= 10

# display the result
if num == sum:
    print(num,"is an Armstrong number")
else:
    print(num,"is not an Armstrong number")
```

Cont.....

- * Here, we ask the user for a number and check if it is an Armstrong number.
- * We need to calculate the sum of the cube of each digit.
- * So, we initialize the sum to 0 and obtain each digit number by using the [modulus operator %](#).
- * The remainder of a number when it is divided by 10 is the last digit of that number.
- * We take the cubes using exponent operator.
- * Finally, we compare the sum with the original number and conclude that it is Armstrong number if they are equal.

Code: Check Armstrong number of n digits

```
num = 1634
```

```
# Changed num variable to string,  
# and calculated the length (number of digits)  
order = len(str(num))
```

```
# initialize sum  
sum = 0
```

```
# find the sum of the cube of each digit  
temp = num  
while temp > 0:  
    digit = temp % 10  
    sum += digit ** order  
    temp //= 10
```

```
# display the result  
if num == sum:  
    print(num,"is an Armstrong number")  
else:  
    print(num,"is not an Armstrong number")
```

Question

Complete the following program to check whether a number is an Armstrong number or not.
What is Armstrong Number? It is a number that is equal to the sum of its digits powered by the total number of digits in the number.
For example: 153, 370, 1634, 8208, 9474 etc.
Suppose you are taking 153 for an example

- First count the number of digits in 153. Since it contains 3 digits calculate the cube of its each digits.
 $1^3 = (1*1*1) = 1$, $5^3 = (5*5*5) = 125$, $3^3 = (3*3*3) = 27$
Now add the cube $1+125+27 = 153$ It means 153 is an Armstrong number.

```
digits=0
result=0
number = 370
number1 = number
temp = number
#the code below is used to count the digits
while number!=0:
    number = _____
    digits += 1
#the code below is used to calculate the sum of each digit in thenumber #powered to the
digit count
while _____:
    number=_____
    result=_____
    number1=number1//10
if _____:
    print("number is armstrong")
else:
    print("number is not armstrong")
```

Answer

python digits counting code

Python Program to Count Number of Digits in a Number using While loop

```
Number = int(input("Please Enter any Number: "))
Count = 0
while(Number > 0):
    Number = Number // 10
    Count = Count + 1
print("\n Number of Digits in a Given Number = %d"
%Count)
```

Sum of each digit powered to number of digit

Question8

calculate number of occurrence of a given character from given string

```
string=input("please enter string: ")
cha=input("please enter character: ")
index , count = 0,0
while(index<len(string)):
    if( cha==string[index])
        count+=1
    index+=1
Print("The total number of occurrence of",
char,"is",count)
```

Python Program to Count Occurrence of a Character in a String

- Write a Python Program to Count Occurrence of a Character in a String with a practical example.
- This python program allows you to enter a string and a character.

for loop

Python Program to Count Occurrence of a Character in a String

```
string = input("Please enter your own String : ")
char = input("Please enter your own Character : ")
count = 0
for i in range(len(string)):
    if(string[i] == char):
        count = count + 1
print("The total Number of Times ", char, " has Occurred = ", count)
```

Cont....

- ✓ Here, we used [For Loop](#) to iterate each character in a [String](#).
- ✓ Inside the [Python](#) For Loop, we used the [If statement](#) to check whether any character in a string is equal to the given character or not.
- ✓ If true, then `count = count + 1`.

Python Program to Count Occurrence of a Character method

2

- This Python counting total occurrence of a character in a string program is the same as the above.
- we just replaced the For Loop with While Loop.

```
# Python Program to Count Occurrence of a Character in a String
```

```
string = input("Please enter your own String : ")
```

```
char = input("Please enter your own Character : ")
```

```
i = 0
```

```
count = 0
```

```
while(i < len(string)):
```

```
    if(string[i] == char):
```

```
        count = count + 1
```

```
    i = i + 1
```

```
print("The total Number of Times ", char, " has Occurred = ", count)
```

Python Program to Count Total Occurrence of a Character method 3

- This Python total occurrence of a given Character program is the same as the first example.
- But, this time, we used the Functions concept to separate the logic.

Python Program to Count Occurrence of a Character in a String

```
def count_Occurrence(ch, str1):  
    count = 0  
    for i in range(len(string)):  
        if(string[i] == char):  
            count = count + 1  
    return count  
string = input("Please enter your own String : ")  
char = input("Please enter your own Character : ")  
cnt = count_Occurrence(char, string)  
print("The total Number of Times ", char, " has Occurred = " , cnt)
```

Part four

check palindrome(string) 5 easy way's

- * In this tutorial, you'll learn how to use Python to check if a string is a palindrome.
- * You'll learn 5 different ways, including using **string indexing**, **for loops**, **the reversed() function**.
- * You'll also learn the limitations of the different approaches, and when one might be better to use than another.
- * But before we dive in, let's answer a quick question:
- * what is a palindrome?
- * *a palindrome is a word, phrase, or sequence that is the same spelled forward as it is backwards*

1. Use String Indexing

- * One of the easiest ways to use Python to check if a string is a palindrome is to use string indexing.
- * One of the greatest things about Python indexing is that you can **set a step counter as well, meaning you can move over an iterable at a desired rate.**
- * The way that we'll make use of this is to use a step of -1, meaning that we move over the iterable from the back to the front.
- * An important thing to keep in mind is that your string may have different casing as well as spaces. Because of this, we'll need to preprocess our string to remove capitalization and any spaces.
- * Let's take a look at using string indexing to check if a string is a palindrome in Python:

code

Use String Indexing in Python to check if a String is a Palindrome

```
a_string = 'Was it a car or a cat I saw'
```

```
def palindrome(string):
```

```
    string = string.lower().replace(' ', '')
```

```
    return string == string[::-1]
```

```
print(palindrome(a_string))
```

steps

Let's explore what we've done here:

1. We define a function that takes a single string
2. We then re-assign the string to itself, but lower all the character cases and remove any spaces
3. We then return the evaluation of whether the string is equal to itself, backwards
4. This returns True if the string is a palindrome, and False if it is not

2. Use the Python Reversed Function to Check if a String is a Palindrome

- Python comes with a built-in function, `reversed()`, which, well, reverses an item.
- You can pass in some iterable item, be it a string, a list, or anything else ordered, and the function returns the reversed version of it.

Use the Reversed() function in Python to check if a String is a Palindrome

`a_string = 'Was it a car or a cat I saw'`

`def` `palindrome(string):`

`string = string.lower().replace(' ', '')`

`reversed_string = ''.join(reversed(string))`

`return` `string == reversed_string`

`print(palindrome(a_string))`

Returns: True

Cont..

Let's take a look at what our function does:

- Similar to the method above, it first changes all casing to lower case and removes all spaces
- Then, it uses the `reverse()` function to reverse the string. This is then joined.
- Finally, the two strings are evaluated against whether or not they're equal.

3.Using a For Loop to Check if a Python String is a Palindrome

♥ Similar to the above methods, we can use a Python for-loop to loop over a string through the reverse to see if its characters match.

Let's see how we can use **a for loop** to check if a string is a palindrome:

```
a_string = 'Was it a car or a cat I saw'
def palindrome(string):
    string = string.lower().replace(' ', '')
    reversed = ''
    for i in range(len(string), 0, -1):
        reversed += string[i-1]
    return string == reversed
print(palindrome(a_string))
```

Cont..

- * What we've done here is traversed the list from the last index, -1, to its first, 0.
- * We then assign that value to a string reversed.
- * Finally, we evaluate whether the two strings are equal.
- * In the next section, you'll learn how to use a Python while loop to see if a palindrome exists.

4. Using a Python While Loop to Check if a String is a Palindrome

- In this section, let's explore how to use a Python while loop to see if a string is a palindrome or not.
- One of the benefits of this approach is that **we don't actually need to reassign a reversed string**, which, if your strings are large, won't consume much memory.

```
a_string = 'Was it a car or a cat I saw'
def palindrome(string):
    string = string.lower().replace(' ', '')
    first, last = 0, len(string) - 1
    while(first < last):
        if(string[first] == string[last]):
            first += 1
            last -= 1
        else:
            return False
    return True
print(palindrome(a_string))
```

Cont..

- What we do here is loop over the string from beginning to end.
- Our while loop evaluates whether or not the i^{th} index from the front is equal to the i^{th} index from the back.
- **A major performance benefit here can be that if a string is clearly not a palindrome, say if the first and last characters don't match, then the loop breaks.**
- **This saves us significant memory and time.**

5. Check if a Number is a Python Palindrome

- ✓ The easiest way to check if a number is a Python palindrome, is **to convert the number to a string and apply any of the methods mentioned above.**
- ✓ Let's see how we can do this using the string indexing method:

Use String Indexing in Python to check if a Number is a Palindrome

```
a_number = 123454321
def palindrome(number):
    number = str(number)
    return number == number[::-1]
print(palindrome(a_number))
```